ASSIGNMENT-11.5

NAME: HABEEBA KHANAM

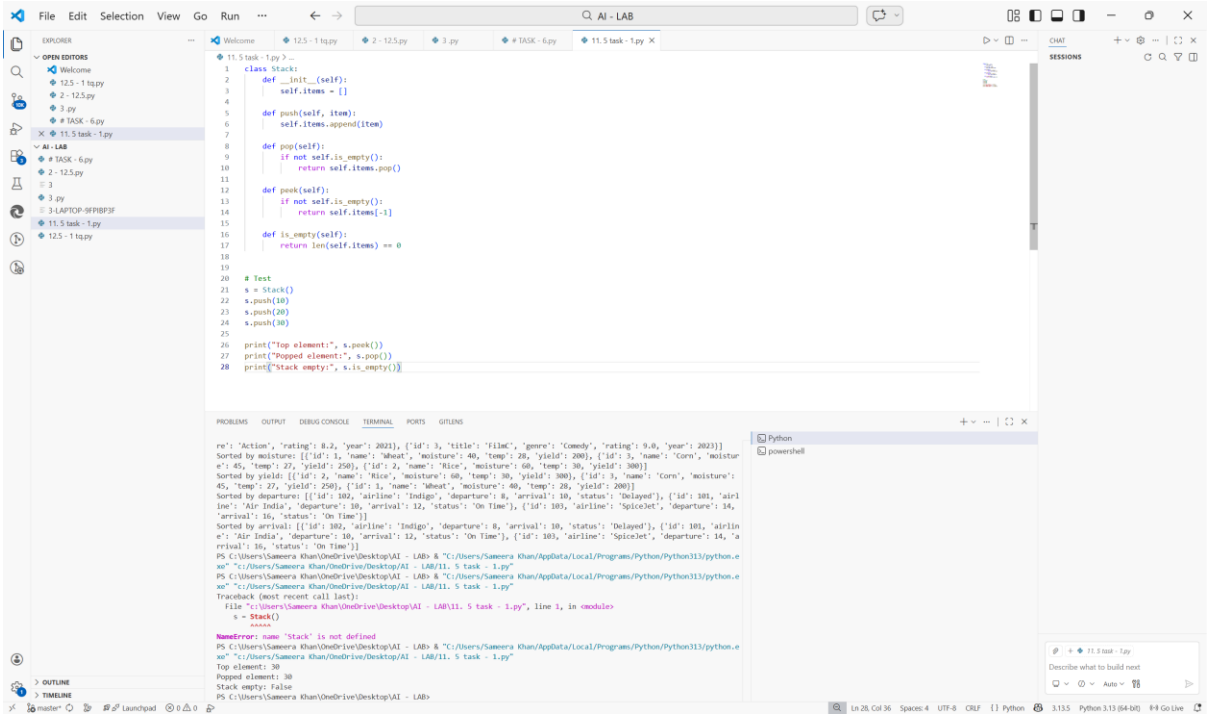
HALL TICKET :2303A51474

BATCH NO:29

LAB 11 – DATA STRUCTURES WITH AI: IMPLEMENTING FUNDAMENTAL STRUCTURES

Task Description #1 – Stack Implementation

CODE:

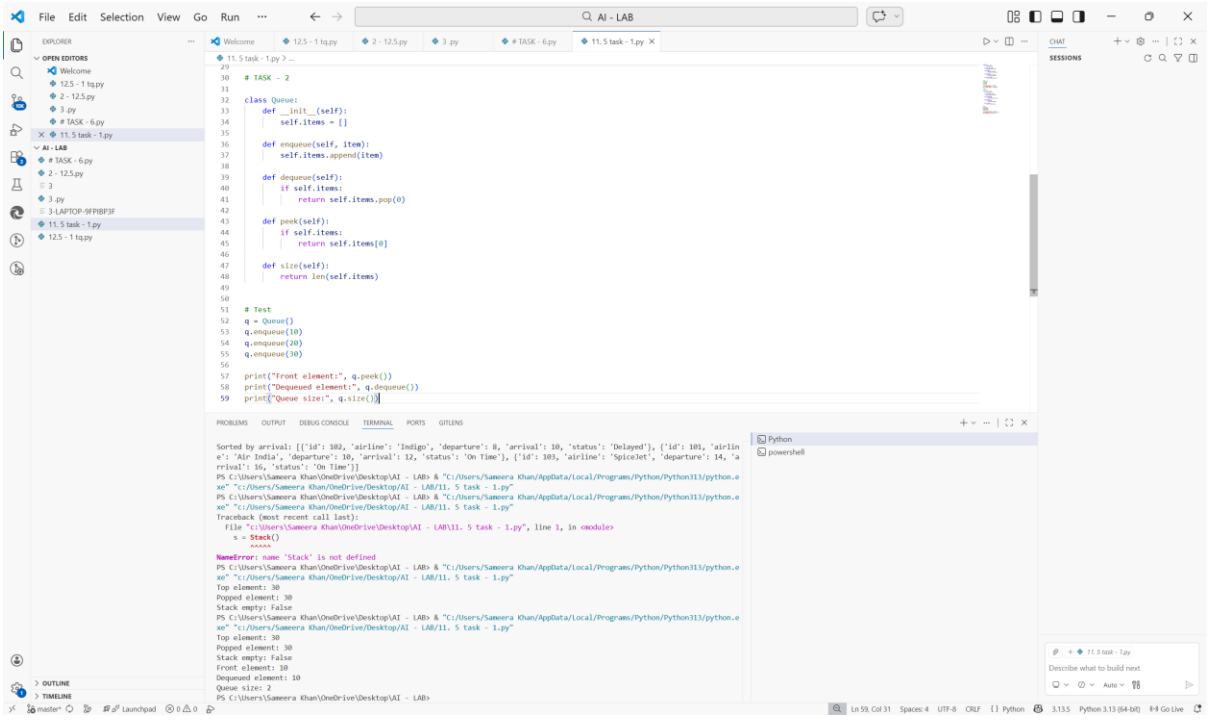


OBSERVATION:

The stack program worked correctly using the LIFO principle. Elements were pushed and popped from the top of the stack. The AI-generated code helped in implementing push, pop, peek, and is_empty methods properly.

Task Description #2 – Queue Implementation

Code:

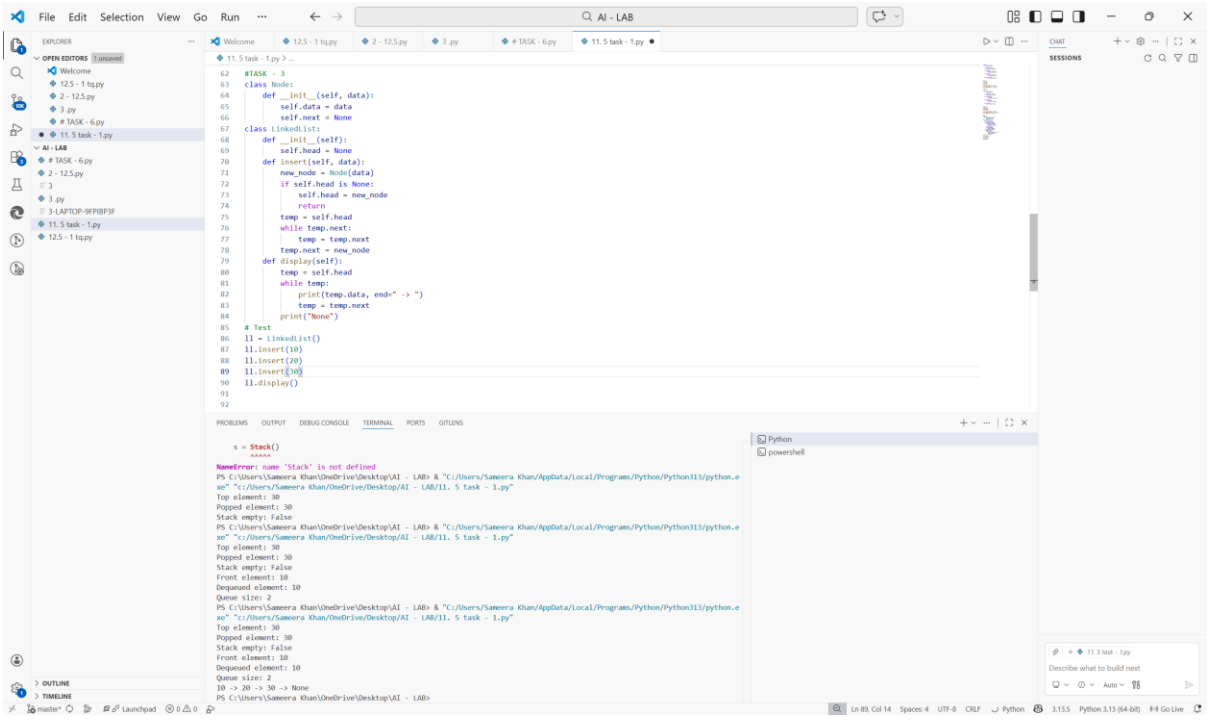


Observation:

The queue program followed the FIFO principle successfully. Elements were inserted using enqueue and removed using dequeue. The implementation correctly maintained the order of elements.

TASK DESCRIPTION #3 – LINKED LIST

CODE:

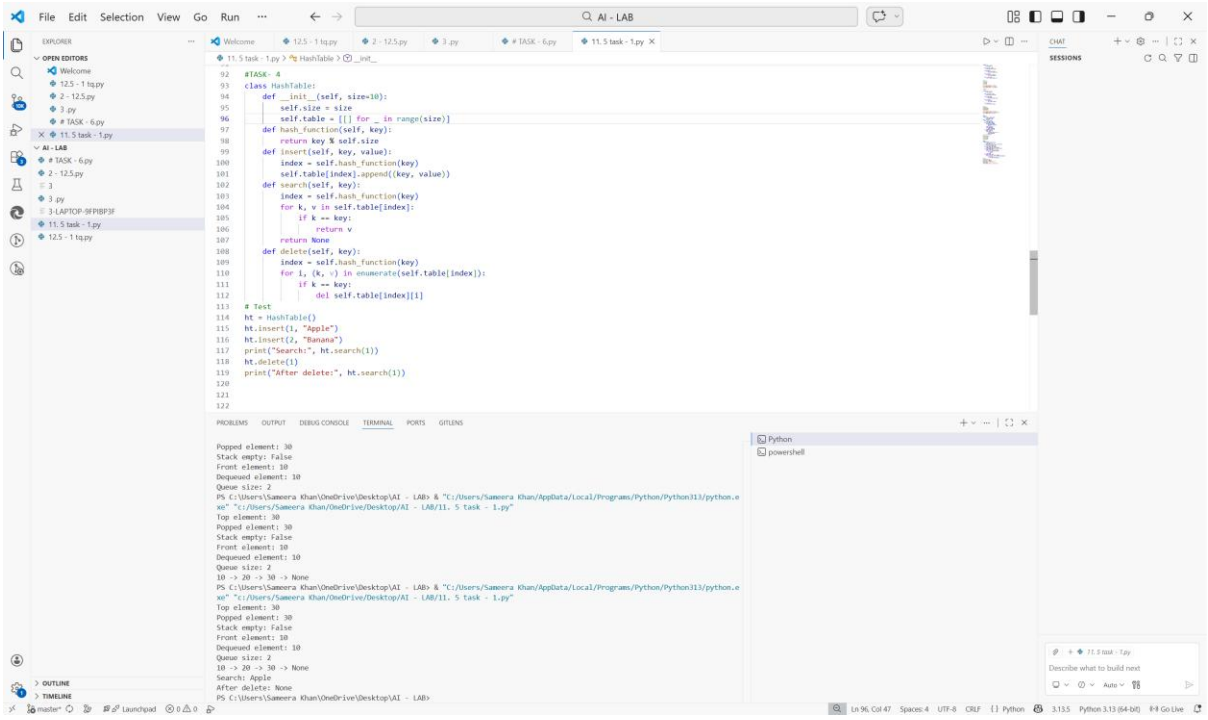


Observation:

The singly linked list program allowed insertion of nodes and displayed the list correctly. Each node was connected through the next pointer. The output confirmed that the linked list structure worked as expected.

TASK DESCRIPTION #4 – HASH TABLE

CODE:



```python
#TASK- 4
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]
    def hash_function(self, key):
        return key % self.size
    def insert(self, key, value):
        index = self.hash_function(key)
        self.table[index].append((key, value))
    def search(self, key):
        index = self.hash_function(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None
    def delete(self, key):
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
# Test
ht = HashTable()
ht.insert(1, "Apple")
ht.insert(2, "Banana")
print("Search:", ht.search(1))
ht.delete(1)
print("After delete:", ht.search(1))
```

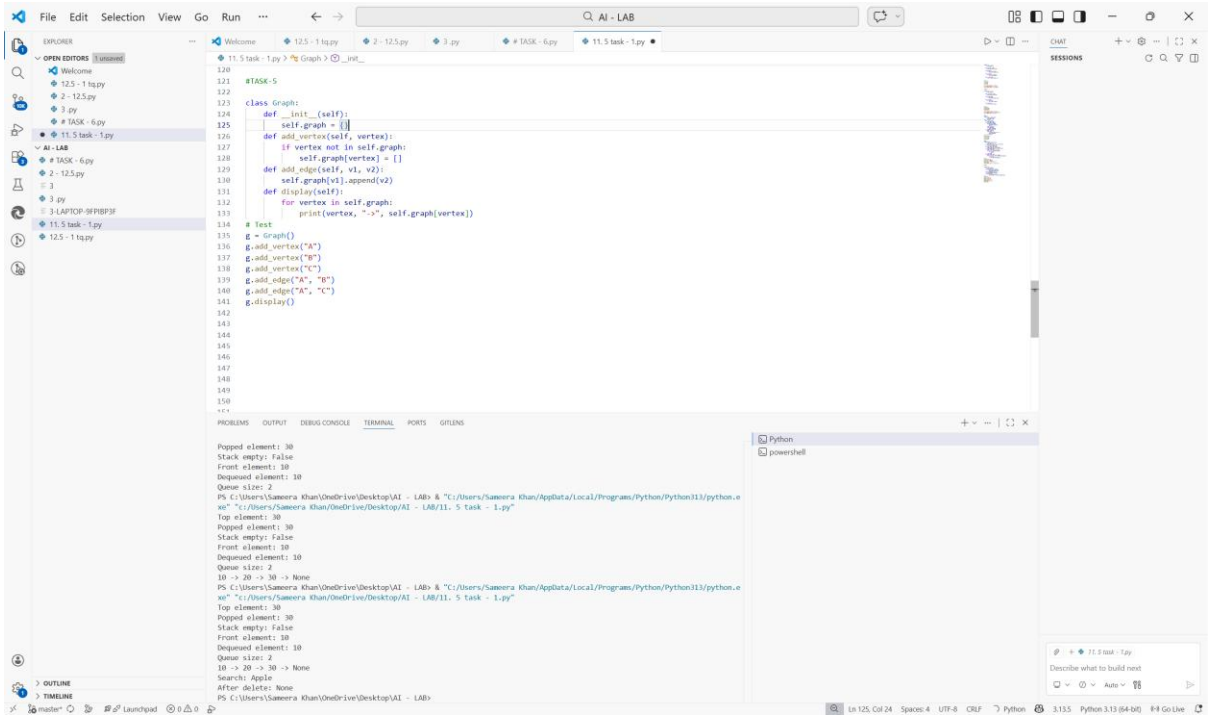
```
Popped element: 30
Stack empty: False
Front element: 10
Dequeued element: 10
Queue size: 2
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB> & "C:/Users/Sameera Khan/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Sameera Khan/OneDrive/Desktop/AI - LAB/11. 5 task - 1.py"
Top element: 30
Popped element: 30
Stack empty: False
Front element: 10
Dequeued element: 10
Queue size: 2
10 -> 20 -> 30 -> None
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB> & "C:/Users/Sameera Khan/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Sameera Khan/OneDrive/Desktop/AI - LAB/11. 5 task - 1.py"
Top element: 30
Popped element: 30
Stack empty: False
Front element: 10
Dequeued element: 10
Queue size: 2
10 -> 20 -> 30 -> None
Search: Apple
After delete: None
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB>
```

OBSERVATION:

The hash table stored and retrieved values using keys efficiently. Collision handling was managed using chaining. The program successfully performed insert, search, and delete operations.

TASK DESCRIPTION #5 – GRAPH REPRESENTATION

CODE:

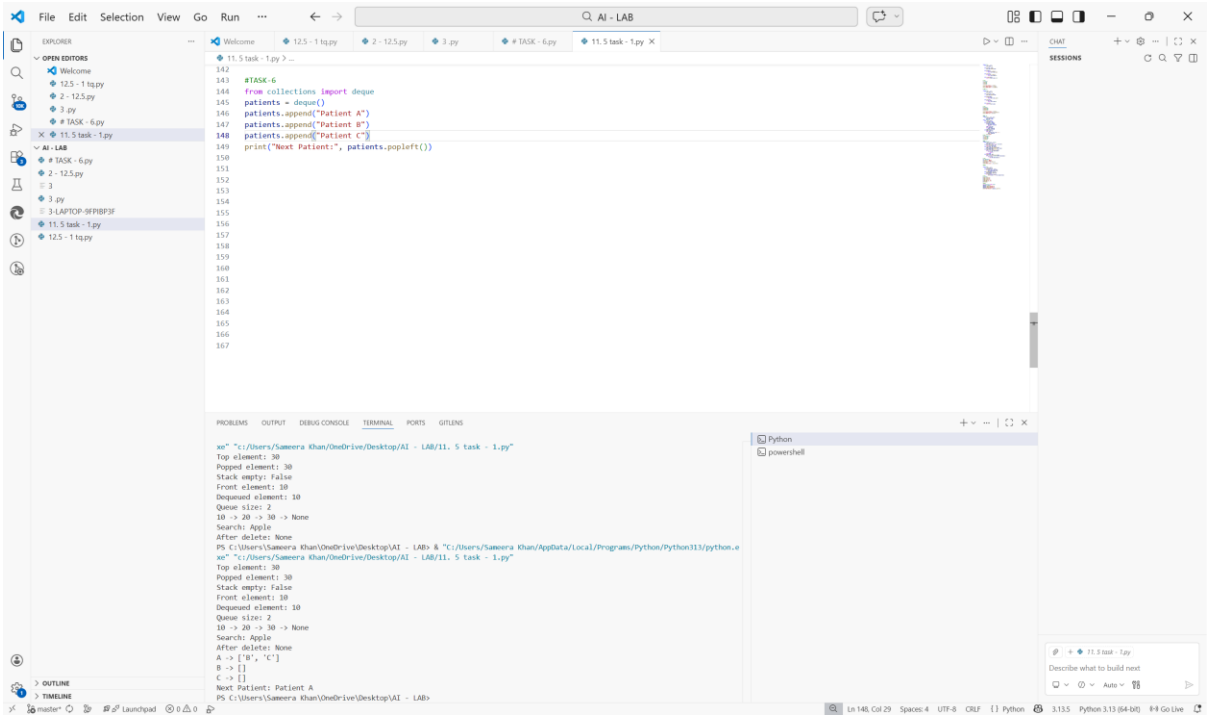


OBSERVATION:

The graph was implemented using an adjacency list representation. Vertices and edges were added successfully. The display function correctly showed connections between nodes.

TASK DESCRIPTION #6: SMART HOSPITAL MANAGEMENT SYSTEM – DATA STRUCTURE SELECTION

CODE:

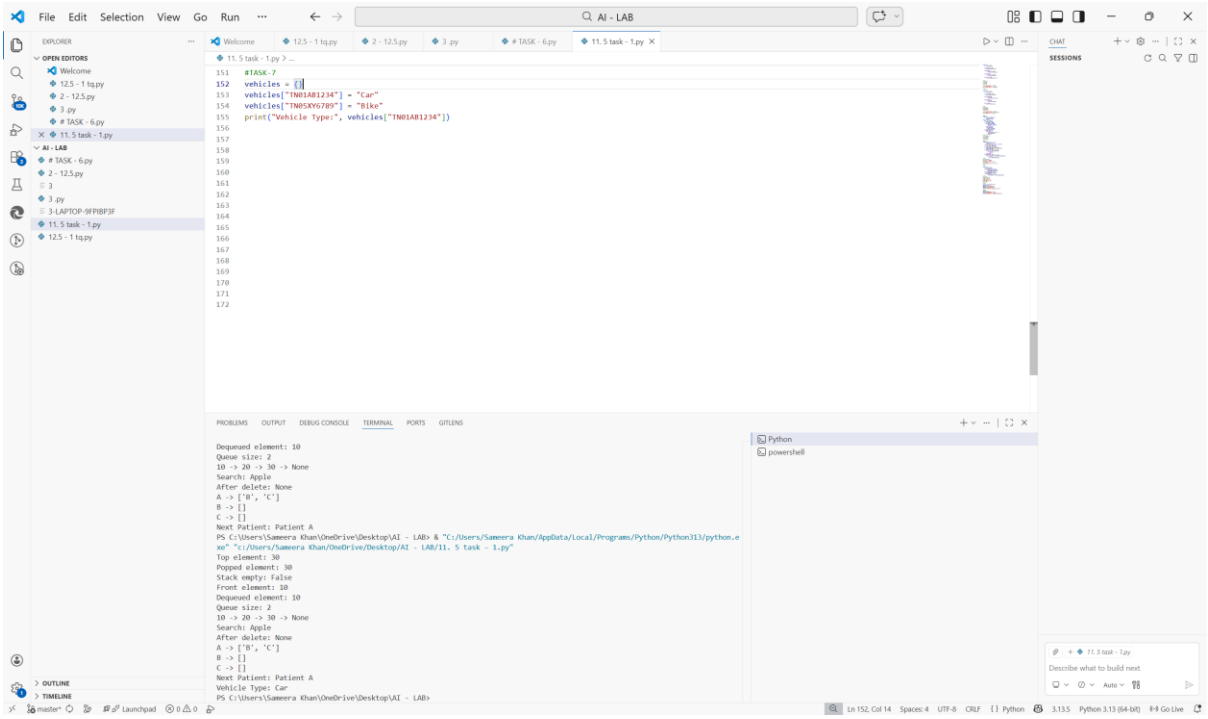


OBSERVATION:

Different data structures were selected for different hospital operations. Queue handled patient check-in, priority queue managed emergency cases, and hash table stored medical records. The chosen structures improved efficiency in managing hospital data.

TASK DESCRIPTION #7: SMART CITY TRAFFIC CONTROL SYSTEM

CODE:

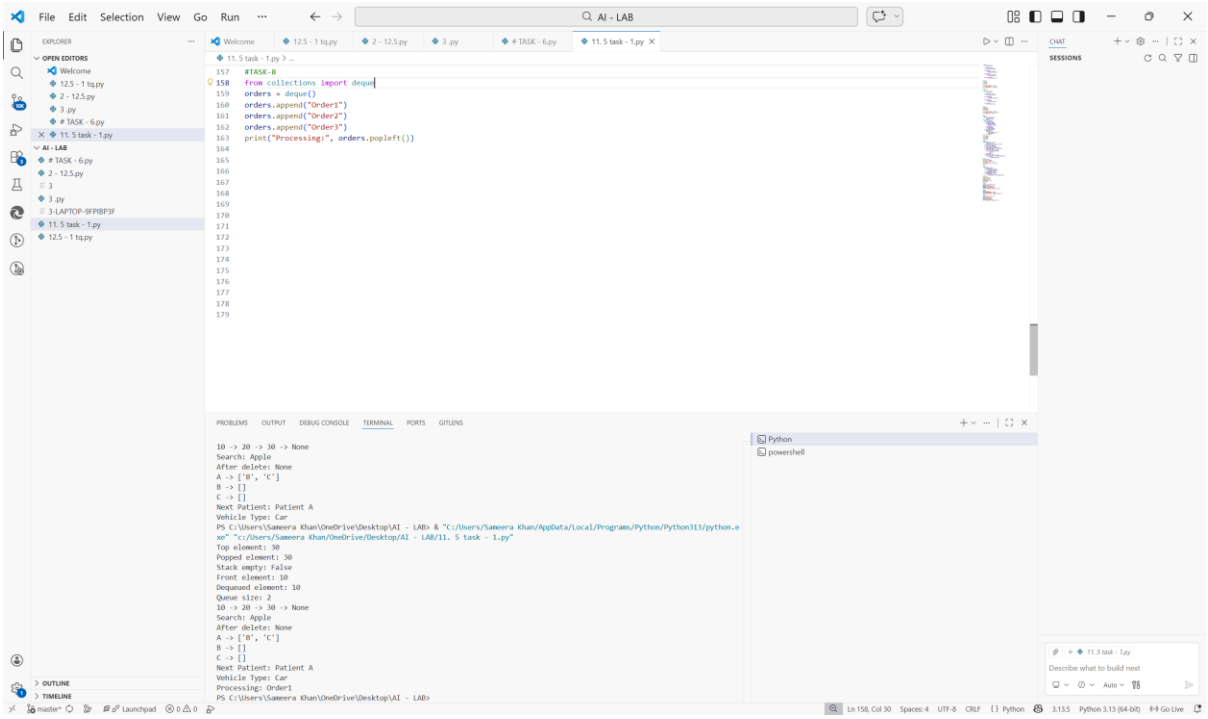


OBSERVATION:

The system used queue for traffic signals and priority queue for emergency vehicles. Hash tables helped in quick vehicle lookup, and graphs represented road networks. These structures helped organize traffic data efficiently.

TASK DESCRIPTION #8: SMART E-COMMERCE PLATFORM – DATA STRUCTURE

CODE:



OBSERVATION:

Different data structures were applied for e-commerce operations. Linked lists managed shopping carts, queues processed orders, and hash tables enabled fast product searches. Graphs helped represent delivery routes effectively.